

State Management

How to wrap it up in my app?

Ephemeral vs global

- Local
- Has my button been touched?
- Short-living in terms of a feature
- Selected tab in bottom navigation
- Current page in the IndexedStack

- Global
- Am I signed in?
- Long-living in terms of a feature, business process or the whole app
- User Authentication state
- Cart items state

We already know about ephemeral state

Ephemeral state

```
class _MyHomePageState extends State<MyHomePage> {
  int _counter = 0;

  void _incrementCounter() {
    setState(() {
      _counter++;
    });
  }

  @override
  Widget build(BuildContext context) {
    return Scaffold(
      appBar: AppBar(
        title: Text(widget.title),
      ),
      body: Center(
        child: Column(
          mainAxisAlignment: MainAxisAlignment.center,
          children: [
            const Text(
              'You have pushed the button this many times:',
            ),
            Text(
              '$_counter',
              style: Theme.of(context).textTheme.headline4,
            ),
          ],
        ),
      ),
    );
  }
}
```

```
class _DraggableCardState extends State<DraggableCard>
  with SingleTickerProviderStateMixin {
  AnimationController? _controller;
  Alignment _dragAlignment = Alignment.center;
  Animation<Alignment>? _animation;

  void _runAnimation(Offset pixelsPerSecond, Size size) {
    _animation = _controller!.drive(
      AlignmentTween(
        begin: _dragAlignment,
        end: Alignment.center,
      ),
    );
  }

  final unitsPerSecondX = pixelsPerSecond.dx / size.width;
  final unitsPerSecondY = pixelsPerSecond.dy / size.height;
  final unitsPerSecond = Offset(unitsPerSecondX, unitsPerSecondY);
  final unitVelocity = unitsPerSecond.distance;

  const spring = SpringDescription(
    mass: 30,
    stiffness: 1,
    damping: 1,
  );

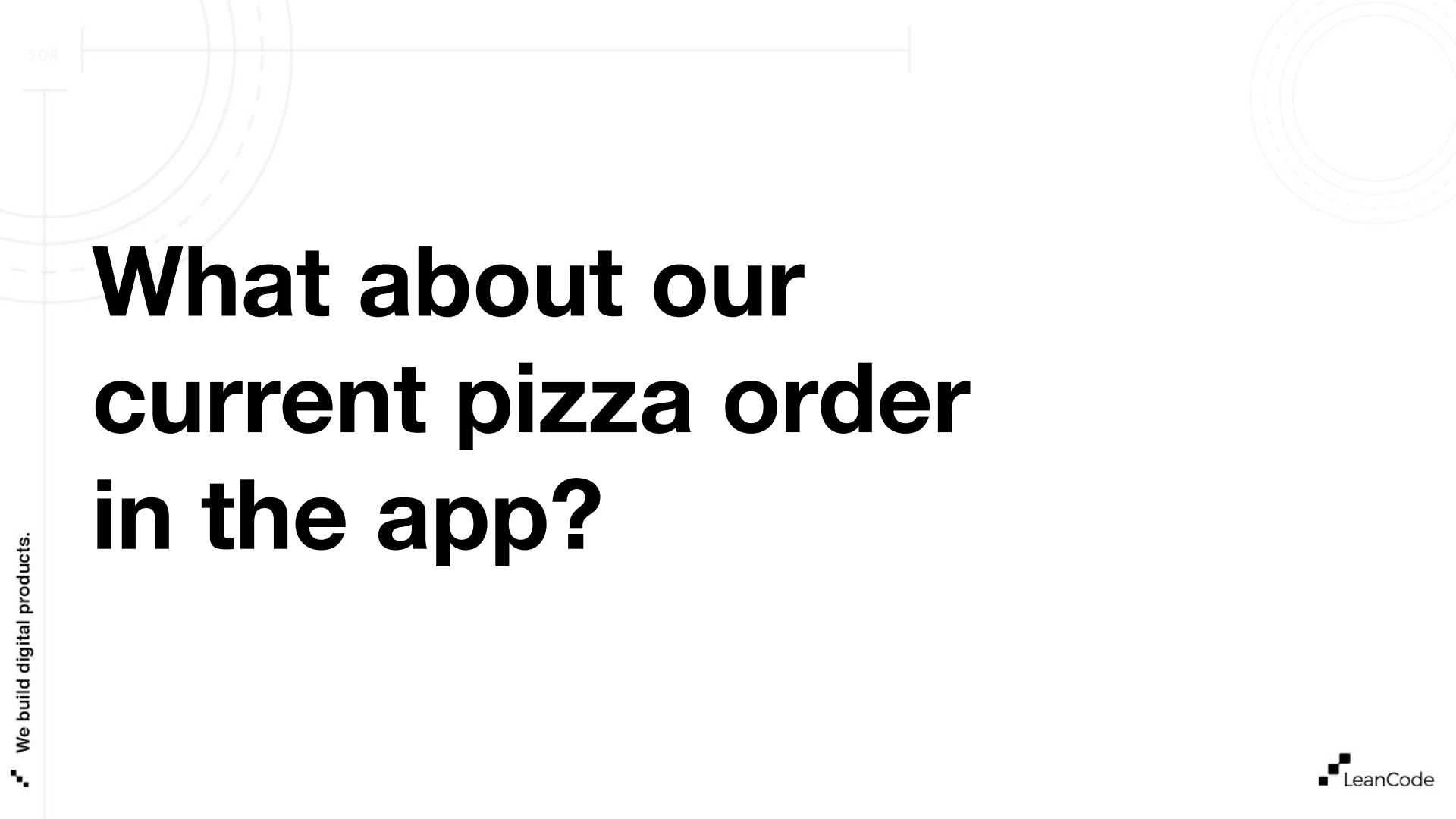
  final simulation = SpringSimulation(spring, 0, 1, -unitVelocity);

  _controller!.animateWith(simulation);
}

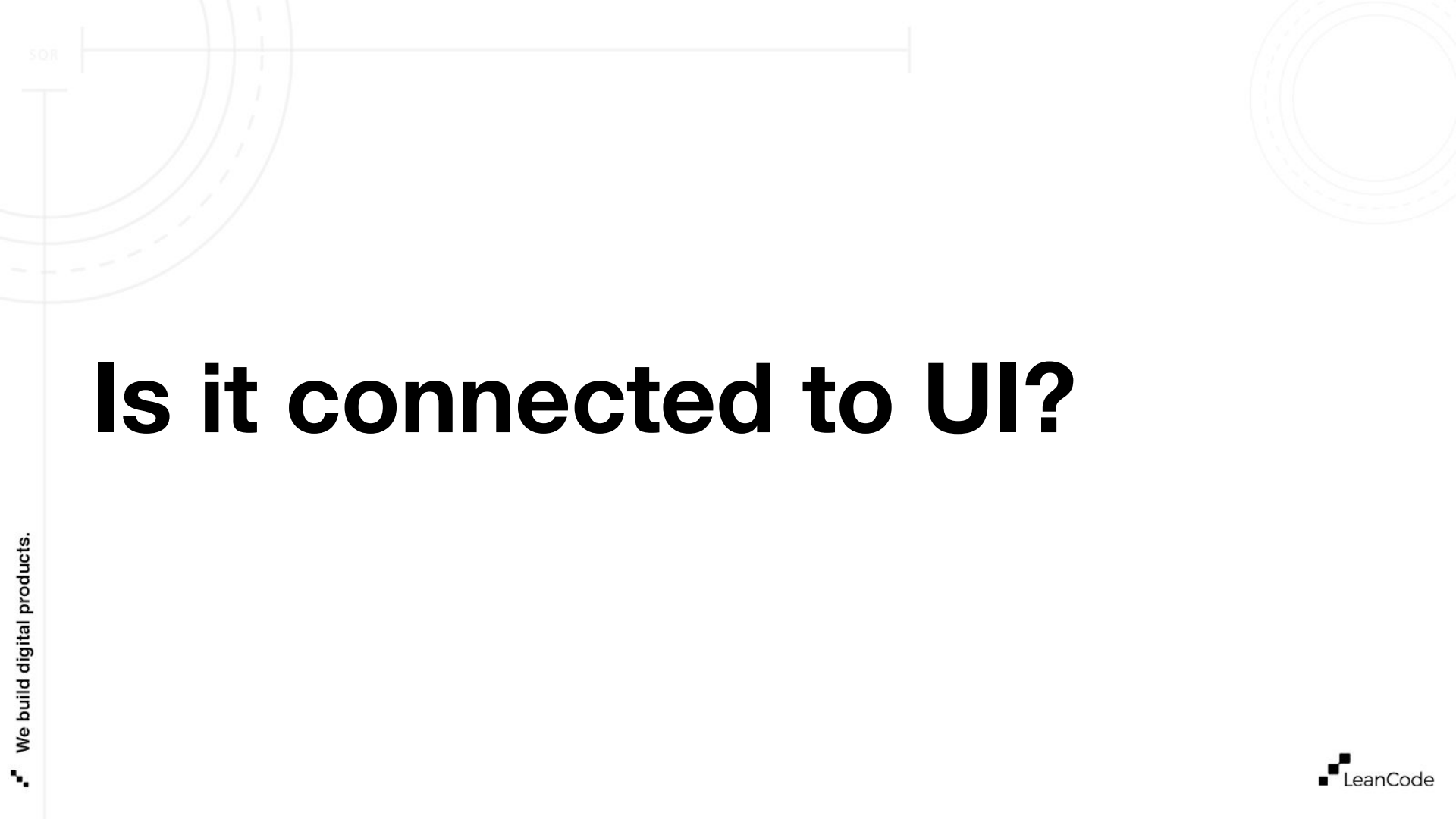
@override
void initState() {
  super.initState();
  _controller = AnimationController(vsync: this);

  _controller!.addListener(() {
    setState(() {
      _dragAlignment = _animation!.value;
    });
  });
}
```

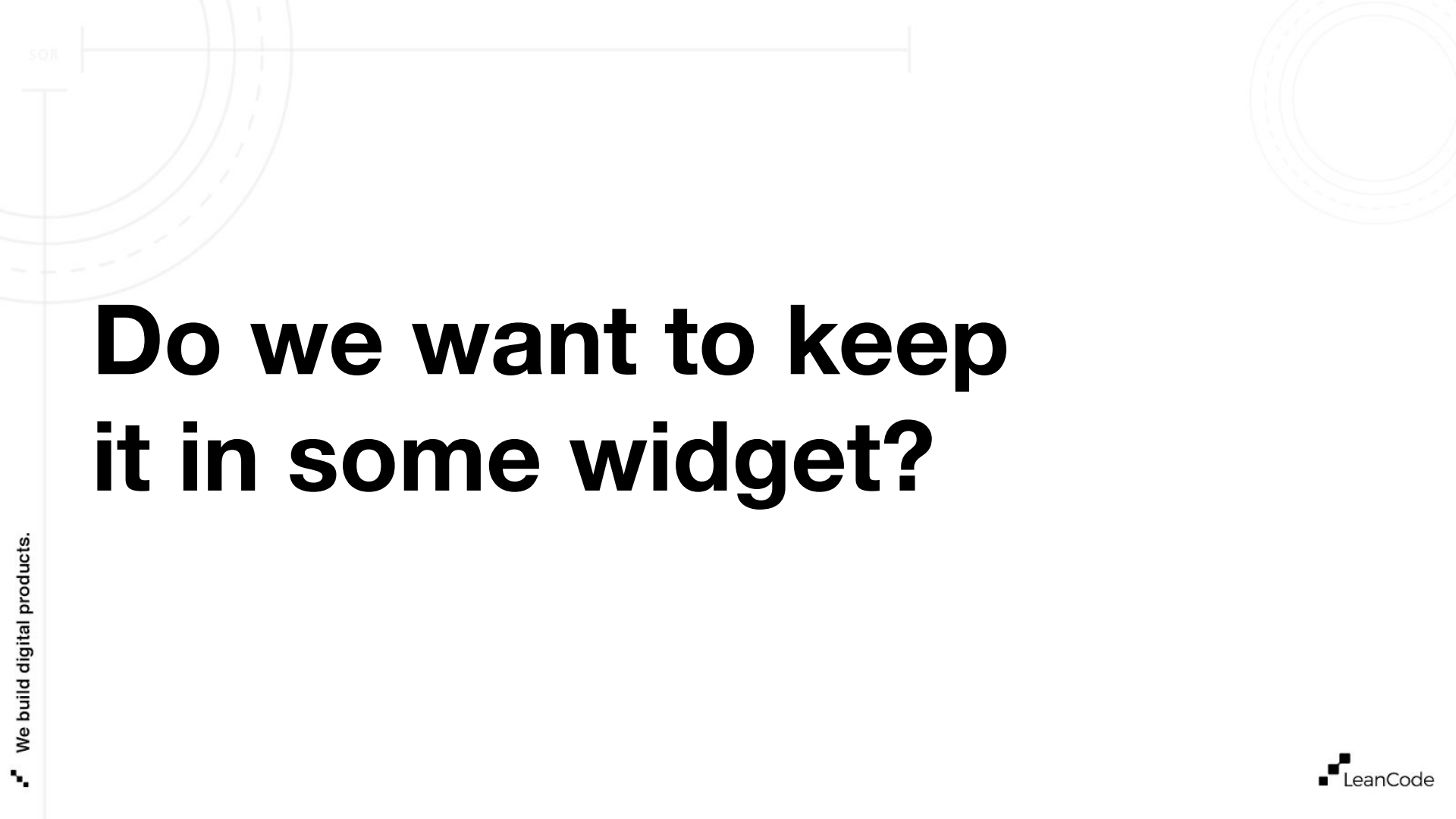
Things that are tightly coupled with our UI



What about our current pizza order in the app?



Is it connected to UI?



**Do we want to keep
it in some widget?**

What about text field?

What about text field in three page form?

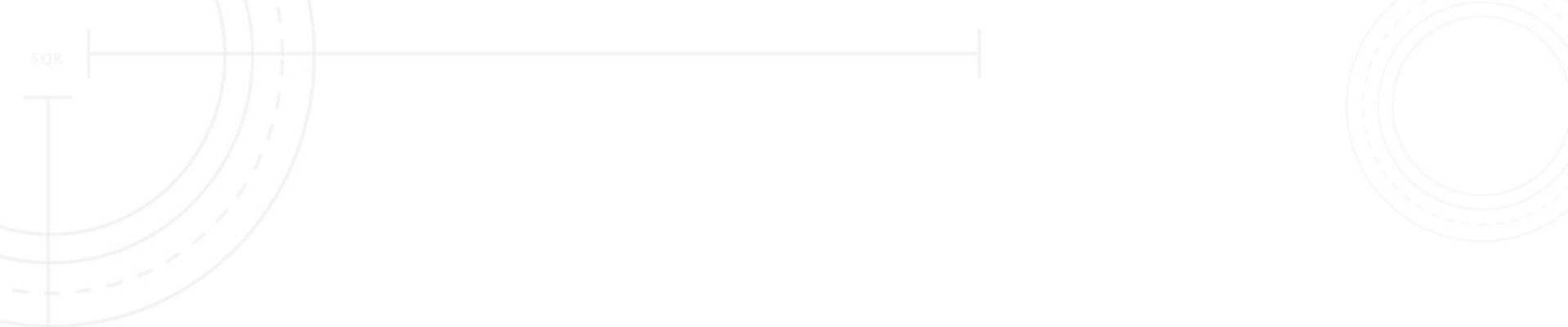
ChangeNotifier

```
class Counter with ChangeNotifier {  
  int _count = 0;  
  
  int get count => _count;  
  
  void increment() {  
    _count++;  
    notifyListeners();  
  }  
}
```

```
floatingActionButton: FloatingActionButton(  
  onPressed: () => context.read<Counter>().increment(),  
  child: const Icon(Icons.add),  
),
```

```
MultiProvider(  
  providers: [  
    ChangeNotifierProvider(create: (context) => Counter()),  
  ],  
  child: const MyApp(),  
),
```

```
class Count extends StatelessWidget {  
  const Count({Key? key}) : super(key: key);  
  
  @override  
  Widget build(BuildContext context) {  
    return Text(  
      '${context.watch<Counter>().count}',  
      key: const Key('counterState'),  
      style: Theme.of(context).textTheme.headline4,  
    );  
  }  
}
```

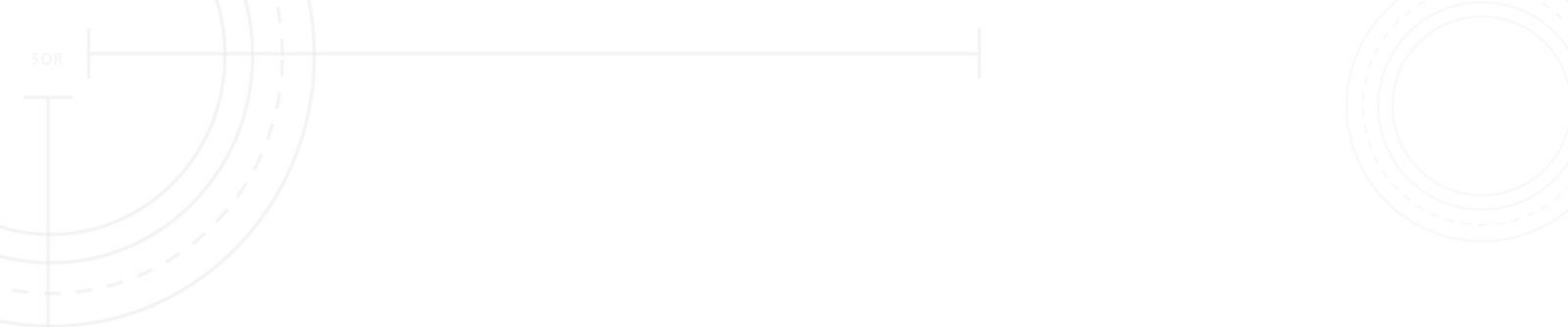


Pretty simple, huh?

**It's just a class so we
can do whatever we
like**

What if...?

```
class Counter with ChangeNotifier {  
  int _count = 0;  
  
  int get count => _count;  
  
  void increment() {  
    _count++;  
  }  
}
```

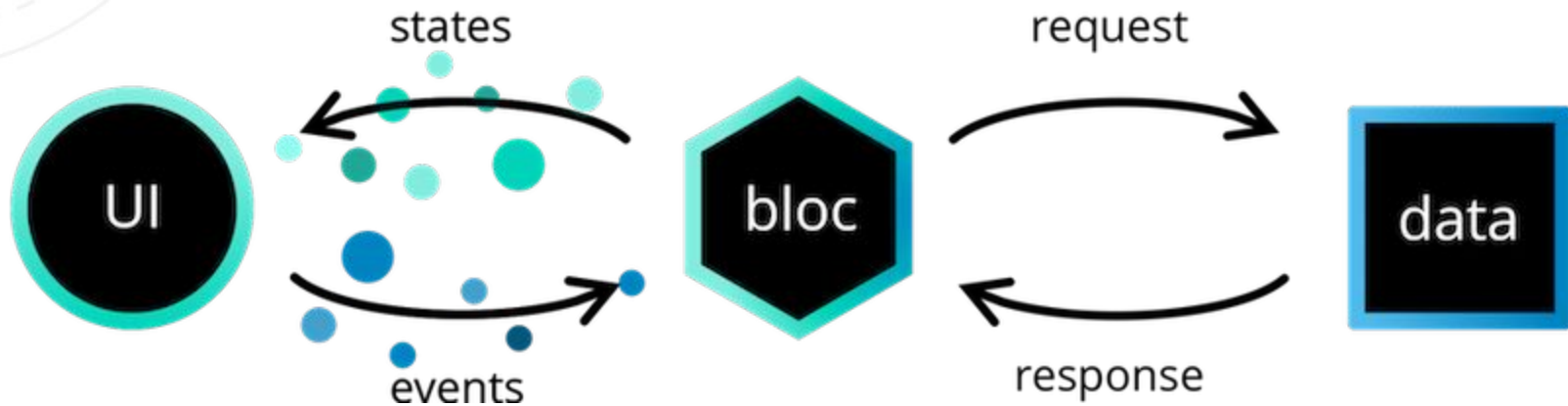


Doesn't standardize

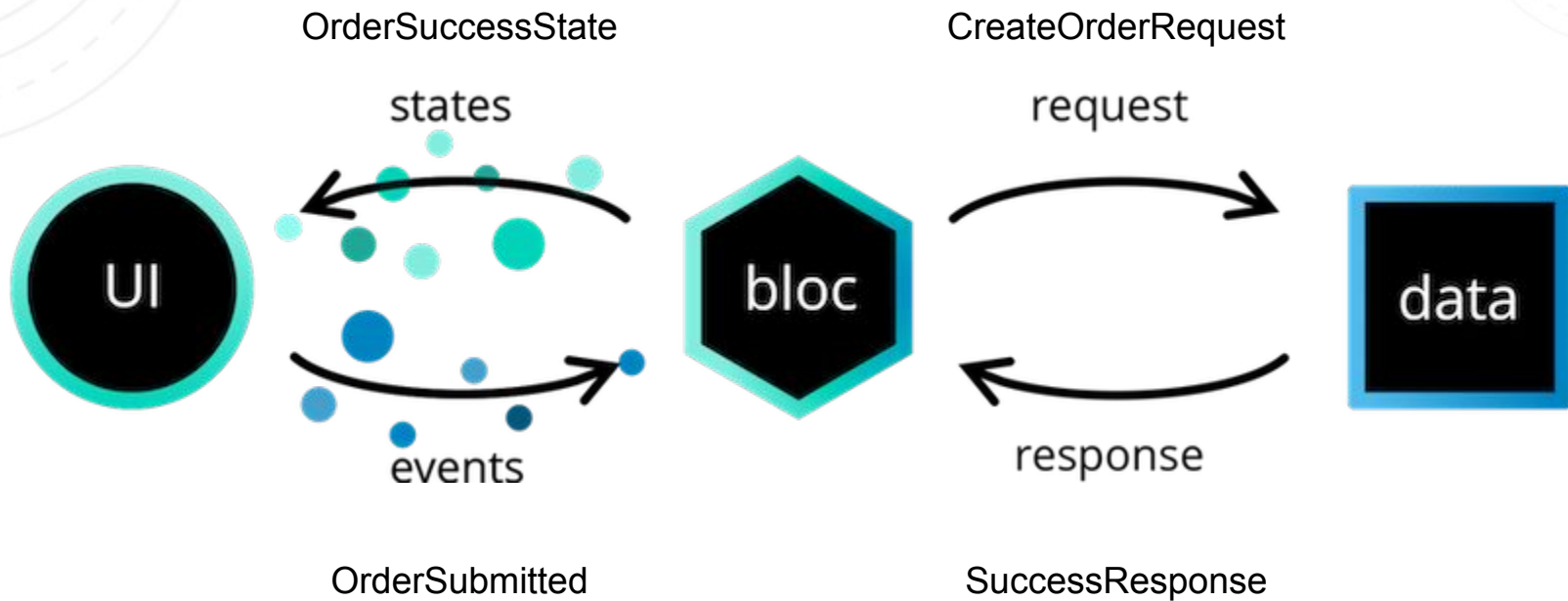
10 devs 100 solutions


```
class ValueNotifier<T> extends ChangeNotifier implements ValueListenable<T> {  
  /// Creates a [ChangeNotifier] that wraps this value.  
  ValueNotifier(this._value);  
  
  /// The current value stored in this notifier.  
  ///  
  /// When the value is replaced with something that is not equal to the old  
  /// value as evaluated by the equality operator ==, this class notifies its  
  /// listeners.  
  @override  
  T get value => _value;  
  T _value;  
  set value(T newValue) {  
    if (_value == newValue)  
      return;  
    _value = newValue;  
    notifyListeners();  
  }  
  
  @override  
  String toString() => '${describeIdentity(this)}($value)';  
}
```

Bloc



Bloc



Bloc

```
abstract class CounterEvent {}  
class Increment extends CounterEvent {}  
class Decrement extends CounterEvent {}
```

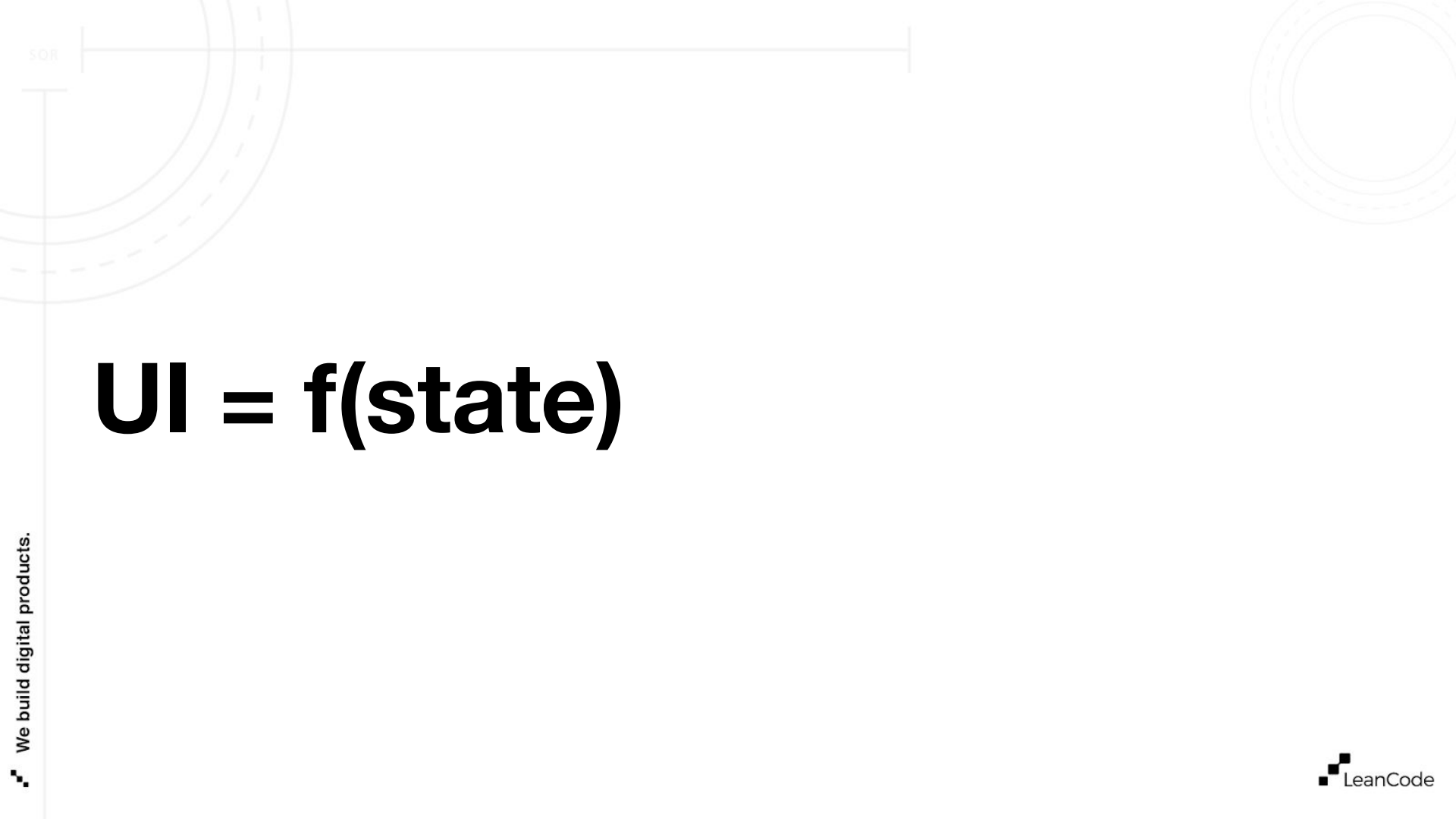
```
class CounterBloc extends Bloc<CounterEvent, int> {  
  CounterBloc() : super(0) {  
    on<Increment>((event, emit) => emit(state + 1));  
    on<Decrement>((event, emit) => emit(state - 1));  
  }  
}
```

Bloc

```
Padding(  
  padding: const EdgeInsets.symmetric(vertical: 5.0),  
  child: FloatingActionButton(  
    child: const Icon(Icons.add),  
    onPressed: () =>  
      context.read<CounterBloc>().add(Increment()),  
  ),  
),
```

```
child: BlocBuilder<CounterBloc, int>(  
  builder: (context, count) {  
    return Text(  
      '$count',  
      style: Theme.of(context).textTheme.headline1,  
    );  
  },  
)
```

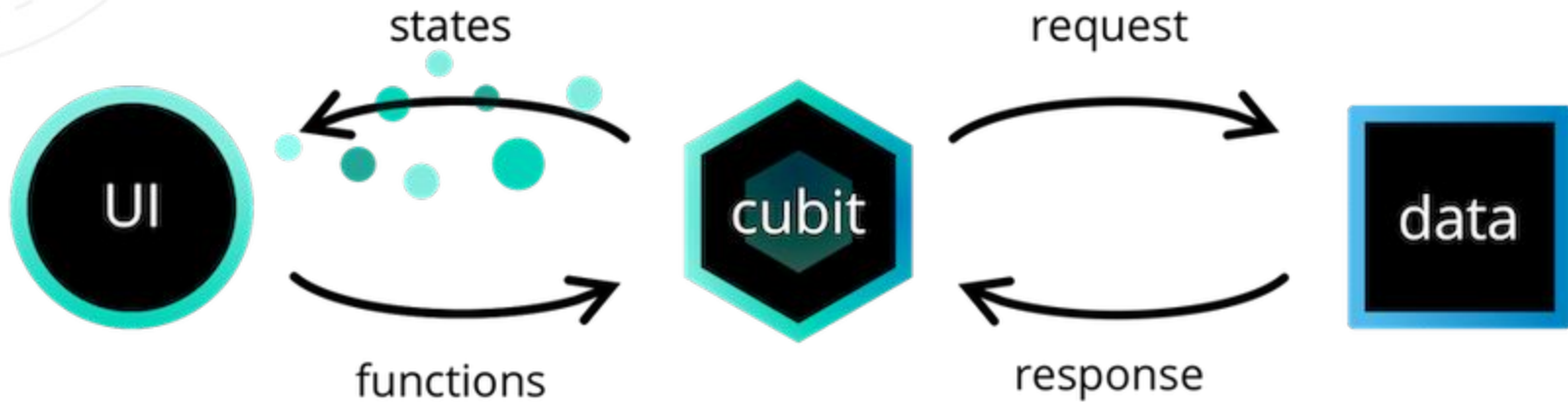
```
return MaterialApp(  
  theme: theme,  
  home: BlocProvider(  
    create: (_) => CounterBloc(),  
    child: CounterPage(),  
  ),  
);
```


$$\text{UI} = f(\text{state})$$

UI = f(state)

```
return BlocBuilder<BaseRequestCubit<dynamic, dynamic, TOut, TError>,
  RequestState<TOut, TError>>{
  bloc: cubit,
  builder: (context, state) {
    return switch (state) {
      InitialState() => const LoadingWidget(),
      LoadingState() => const LoadingWidget(),
      ErrorState() => Column(
        children: [
          Text('Something has gone wrong'),
          TextButton(child: Text('retry'), onPressed: onErrorCallback),
        ],
      ),
      SuccessState(:final data) => builder(context, data),
    };
  },
);
```

Cubit



Cubit

```
class CounterCubit extends Cubit<int> {  
  CounterCubit() : super(0);  
  
  void increment() => emit(state + 1);  
  void decrement() => emit(state - 1);  
}
```

```
sealed class RankingListState {  
    const RankingListState();  
  
    const factory RankingListState.initial() = RankingListStateInitial;  
  
    const factory RankingListState.inProgress() = RankingListStateInProgress;  
  
    const factory RankingListState.ready({required RankingLineDTO? me}) =  
        RankingListStateReady;  
  
    const factory RankingListState.failure() = RankingListStateFailure;  
}
```

```
class RankingListStateInitial extends RankingListState {  
    const RankingListStateInitial();  
}  
  
class RankingListStateInProgress extends RankingListState {  
    const RankingListStateInProgress();  
}  
  
class RankingListStateReady extends RankingListState {  
    const RankingListStateReady({required this.me});  
  
    final RankingLineDTO? me;  
}  
  
class RankingListStateFailure extends RankingListState {  
    const RankingListStateFailure();  
}
```

```
class RankingListCubit extends Cubit<RankingListState> {
  RankingListCubit({required RankingsCqrs rankingsCqrs})
    : _rankingsCqrs = rankingsCqrs,
      super(const RankingListState.initial());

  final RankingsCqrs _rankingsCqrs;

  static final _logger = Logger('RankingListCubit');

  Future<void> fetch() async {
    if (state is RankingListStateInProgress) {
      return;
    }

    emit(const RankingListState.inProgress());

    try {
      final rankingResult = await _rankingsCqrs.get(ContestRanking());
      final ranking = rankingResult.dataOrNull;

      if (ranking == null) {
        _logger.severe('Returned ranking was null');
        emit(const RankingListState.failure());
        return;
      }

      emit(RankingListState.ready(me: ranking.me));
    } catch (e, st) {
      _logger.info('Failed to fetch ranking', e, st);
      emit(const RankingListState.failure());
    }
  }
}
```



“Nail factory” :)

leancode_cubit_utils

```
import 'package:leancode_cubit_utils/leancode_cubit_utils.dart';

class RankingListCubit extends QueryCubit<RankingPageDT0?, RankingLineDT0?> {
  RankingListCubit({required RankingsCqrs rankingsCqrs})
    : _rankingsCqrs = rankingsCqrs,
      super('RankingListCubit');

  final RankingsCqrs _rankingsCqrs;

  @override
  RankingLineDT0? map(RankingPageDT0? data) => data?.me;

  @override
  Future<QueryResult<RankingPageDT0?>> request() =>
    _rankingsCqrs.get(ContestRanking());
}
```

Sources

<https://flutter.dev>

<https://dartpad.dev>

<https://bloclibrary.dev>